PART 8: THE NoSQL LAYERS

Problem Inventory

#	Problem	Severity	Business Impact
8.1	JSONB product_enrichment: 7+ representations of same attribute	High	Wrong analytics (screen size averages wrong, marketing publishing incorrect data)
8.2	GIN index uses jsonb_ops instead of jsonb_path_ops	Medium	72% of queries use containment operators that benefit from path_ops
8.3	Elasticsearch consumer crashed 3 months ago, 14.2M unprocessed events	Critical	Stale search results, PKR 18.4M loss from wrong prices
8.4	No alerting on ES consumer process	High	Crash went undetected for 3 months
8.5	Redis: 4 naming conventions, 4 serialization formats, 4 TTL policies	High	Operational chaos
8.6	Redis: Inventory cache has no TTL (permanent)	Critical	Customers see "In Stock" for out-of-stock items
8.7	Redis: FLUSHDB is only fix, causes 3-4 hour cache storm	High	DB overload twice/week
8.8	Fraud detection CTE: 45-55 min/run, hourly schedule	Critical	PKR 4.2M fraud loss in detection gap
8.9	Fraud detection: no transaction isolation, no graph characterization	High	Unknown if CTE approach is even viable

Part 8: NoSQL Layers & Data Cleansing

- **8.1 Fragmented schemas in JSONB**
    - **Problem:** Unregulated JSON inserts created wildly inconsistent keys (e.g., display_size vs screen_size) and data types, breaking frontend filters.
    - **Solution:** Run migration scripts to normalize attributes to a canonical format and merge duplicate aliases.
    - **Why Chosen:** Standardizes the data to allow reliable filtering while preserving NoSQL flexibility for future dynamic product attributes.

- **8.2 Lack of schema validation**
    - **Problem:** The database accepts malformed JSON, meaning bad data continuously enters the system even after batch cleansing.
    - **Solution:** Implement PL/pgSQL validation functions and database-level CHECK constraints for JSON attributes.
    - **Why Chosen:** Acts as an unbypassable gatekeeper, ensuring that any upstream application bugs fail at the database level rather than corrupting the dataset.
- **8.3 Inefficient default GIN index**
    - **Problem:** The default GIN index on product_enrichment is bloated and slow for containment queries.
    - **Solution:** Drop the default index and replace it with a highly optimized jsonb_path_ops GIN index.
    - **Why Chosen:** jsonb_path_ops is significantly smaller and faster for the @> (containment) operations that dominate the frontend workload.
- **8.4 Stale ElasticSearch and crashed consumers**
    - **Problem:** The application manually syncs ElasticSearch, leading to data drift and outdated search results.
    - **Solution:** Set up a PostgreSQL logical publication to push changes to ElasticSearch via Debezium Change Data Capture (CDC).
    - **Why Chosen:** Eliminates brittle application-level dual-writes by making the database WAL the absolute, automated source of truth for the search index.
- **8.5 Redis missing TTLs and broken caching**
    - **Problem:** Inventory cache keys lack Time-To-Live (TTL), causing stale data reads and unbounded Redis memory growth.
    - **Solution:** Implement strict TTLs and switch to a write-through caching pattern where database updates synchronously refresh Redis.
    - **Why Chosen:** Guarantees memory is freed automatically and ensures the cache is never fatally desynchronized from the database.